Check if 30% reduction: □

Last name: _____

First name: _____

Pers. code: _____

Signature: _____

Databases 2 - exam - July 17, 2023 - **Duration 2h**

S. Comai, P. Fraternali, D. Martinenghi

Students with 30% reduction can omit:
point 4 of the exercise on ranking, point 4 of the exercise triggers, and the code of class Award of the exercise on JPA.

A. Concurrency Control (7 points)

Consider the hierarchy of resources (Root (P1 (t1 t2 t3)) (P2 (t4 t5))). Describe the behaviour of the scheduler given the current status of resources provided below. There are three active transactions and the next sequence of accesses is:

$w_2(t1)$, $r_2(P2)$, $w_1(t5)$, $w_3(t2)$

Each transaction commits just after the last operation in the above sequence: transaction 2 commits just after the execution of $r_2(P2)$, transaction 1 just after $w_1(t5)$ and transaction 3 just after $w_3(t2)$. When a lock can be granted, transactions waiting for resources have a higher priority with respect to the next operations in the sequence. Complete the table below with the next locks.

operation	Root	P1	t1	t2	t3	P2	t4	t5
...	ISL_1, ISL_2, IXL_3	ISL_2, SL_1	SL_2	–	–	IXL_3	XL_3	–

Solution.

operation	Root	P1	t1	t2	t3	P2	t4	t5
	ISL_1, ISL_2, IXL_3	ISL_2, SL_1	SL_2	–	–	IXL_3	XL_3	–
$w_2(t1)$ must wait	ISL_1, IXL_2 **(upgrade)**, IXL_3	ISL_2, $SL_1 \leftarrow IXL_2$ **conflict!!**	SL_2	–	–	IXL_3	XL_3	–
$r_2(P2)$ not done, it is waiting								
$w_1(t5)$	IXL_1 **(upgrade)**, IXL_2, IXL_3	ISL_2, $SL_1 \leftarrow IXL_2$	SL_2	–	–	IXL_3, IXL_1	XL_3	XL_1
T1 commits	IXL_2, IXL_3	ISL_2, IXL_2 can be granted!	SL_2	–	–	IXL_3	XL_3	
$w_2(t1)$	IXL_2, IXL_3	IXL_2 **(upgrade)**	XL_2 (upgrade)	–	–	IXL_3	XL_3	
$r_2(P2)$ must wait	IXL_2, IXL_3	IXL_2	XL_2	–	–	$IXL_3 \leftarrow SL_2$ **conflict!**	XL_3	
$w_3(t2)$	IXL_2, IXL_3	IXL_2, IXL_3	XL_2	XL_3	–	$IXL_3 \leftarrow SL_2$	XL_3	
T3 commits	IXL_2	IXL_2	XL_2	–	–	SL_2 can be granted!	–	
$r_2(P2)$	IXL_2	IXL_2	XL_2	–	–	SL_2	–	
T2 commits	–	–	–	–	–	–	–	

B. Ranking (9 points)

Consider the following rankings x and y over the $[0, 10]$ domain, accessible by both sorted and random access, but only until depth 2 (so, for instance, on y you can make a random access for A, but not for B):

x		y	
B:	8	C:	9
C:	6	A:	7
⋮		⋮	

For each of the scoring functions listed below state whether the top-1 element can be established with a suitable ranking algorithm (among those we saw in class) by checking the appropriate box, and

- if the top-1 element (possibly tied) can be found with certainty, apply the algorithm to find it;
- if not, for <u>each</u> of the elements (A, B, C), show how to extend the rankings up to depth 3 (i.e., adding one row) so that the element in question becomes top-1, if possible.

1. $\max\{x, y\}$ 2. $\min\{x, y\}$ 3. $x + y$ 4. $x \cdot y$

1. $\max\{x, y\}$: ☐ The top-1 element can be found ☐ The top-1 element cannot be found

2. $\min\{x, y\}$: ☐ The top-1 element can be found ☐ The top-1 element cannot be found

3. $x + y$: □ The top-1 element can be found □ The top-1 element cannot be found

4. $x \cdot y$: □ The top-1 element can be found □ The top-1 element cannot be found

Solution.

1. For max, we can certainly find the top-1 element by accessing just the first row by sorted access, which is what B0 would do. In this case, C is the top-1 element with an overall max score of 9.

2. For min, with the available information we can establish (as NRA would) that C's score is 6 and that the upper bounds of A and B are 6 and 7, respectively, while their lower bound is 0:

obj	x	y	lb	ub
C	6	9	6	6
B	8	?	0	7
A	?	7	0	6
τ	6	7	threshold: 6	

Therefore all can be the top-1 element (A only tied) and for this to happen the rankings can be extended as follows:

C is top-1

x	y
⋮	⋮
A: 5	B: 5
⋮	⋮

B is top-1

x	y
⋮	⋮
A: 5	B: 7
⋮	⋮

A is top-1 (tied with C)

x	y
⋮	⋮
A: 6	B: 5
⋮	⋮

3. Similarly, for $x + y$, NRA would find the following bounds and scores, thereby guaranteeing that C is top-1 (B can also be top-1 tied with C if its y score turns out to be 7, while A cannot be top-1):

obj	x	y	lb	ub
C	6	9	15	15
B	8	?	8	15
A	?	7	7	13
τ	6	7	threshold: 13	

4. For $x \cdot y$, NRA would find the following bounds and scores, so C and B can be top-1, while A cannot:

obj	x	y	lb	ub
C	6	9	54	54
B	8	?	0	56
A	?	7	0	42
τ	6	7	threshold: 42	

The rankings can, e.g., be extended as follows:

C is top-1

x	y
⋮	⋮
A: 6	B: 6
⋮	⋮

B is top-1

x	y
⋮	⋮
A: 6	B: 7
⋮	⋮

C. Triggers (9 points)

A database stores information about performers and movies, with the following schema. Primary keys are represented with capital letters.

```
MOVIE (MOVIEID, title, duration, budget)
PERSON (PID, name, total_salary)
PERFORMANCE (MOVIEID, PID, ROLE, salary) -- performer PID played ROLE in movie MOVIEID
RICHEST (PID)
MOST_VERSATILE (PID, MOVIEID, number_of_roles)
```

Design a trigger systems that reacts to insertions in the `PERFORMANCE` table (or to actions of triggers reacting to that) in order to do the following:

1. reject the addition of a performer in a movie if, with the added salary, the movie's budget is exceeded;

2. compute the cumulative salary of all the movies of a performer by updating the `total_salary` attribute of the `PERSON` table (initially set to 0);

3. populate the `RICHEST` table (initially empty) with the `PID` of the performer with the highest cumulative salary (ties broken arbitrarily);

4. maintain the `MOST_VERSATILE` table (initially empty), so that it contains the top-5 `(PID,MOVIEID)` pairs for performers who played the highest number of roles within the same movie (in case of ties on the fifth position or earlier, the table should contain all tied pairs).

Solution.

1. ```
   CREATE TRIGGER limited_budget
   AFTER INSERT ON PERFORMANCE
   FOR EACH ROW
   BEGIN
     IF (SELECT SUM(salary) FROM PERFORMANCE WHERE MOVIEID = NEW.MOVIEID) >
       (SELECT budget FROM MOVIE WHERE MOVIEID = NEW.MOVIEID)
     THEN
       RAISE EXCEPTION 'Budget exceeded';
     END IF;
   END;
   ```

2. ```
   CREATE TRIGGER total_salary
   AFTER INSERT ON PERFORMANCE
   FOR EACH ROW
   BEGIN
     UPDATE PERSON SET total_salary = total_salary + NEW.salary WHERE PID = NEW.PID;
   END;
   ```

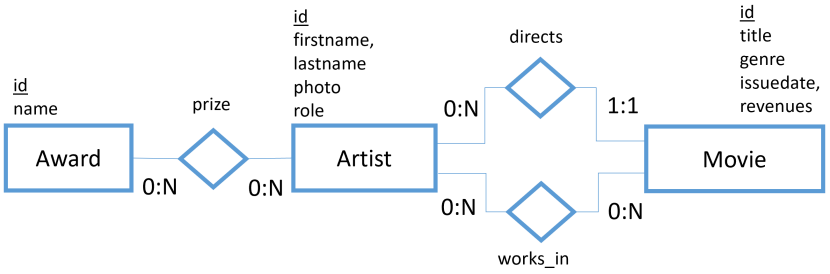
3. ```
   CREATE TRIGGER richest
   AFTER UPDATE OF total_salary ON PERSON
   FOR EACH ROW
   DECLARE
     rich int;
     tot int;
   BEGIN
     SELECT R.PID, P.total_salary into rich, tot FROM RICHEST R, PERSON P
     WHERE R.PID = P.PID;
     IF rich is NULL THEN
       INSERT INTO RICHEST VALUES (NEW.PID);
     ELSE
       IF tot < NEW.total_salary THEN
         UPDATE RICHEST SET PID = NEW.PID;
       END IF;
     END IF;
   END;
   ```

4. ```
   CREATE TRIGGER mv
   AFTER INSERT ON PERFORMANCE
   FOR EACH ROW
   DECLARE
     roles int;
   BEGIN
     SELECT COUNT(*) INTO roles FROM PERFORMANCE
     WHERE PID = NEW.PID AND MOVIEID = NEW.MOVIEID;

     IF -- not yet 5 performers in the top-5
     (SELECT count(*) FROM MOST_VERSATILE) < 5
     OR roles >= -- at least as many roles as the current top-5
       (SELECT number_of_roles FROM MOST_VERSATILE
        ORDER BY number_of_roles
        FETCH FIRST 1 ROW ONLY)
     THEN
       -- remove the entry for that PID,MOVIEID, if it exists
       DELETE FROM MOST_VERSATILE
       WHERE PID = NEW.PID AND MOVIEID = NEW.MOVIEID;

       -- insert the pair in the top-5 list
       INSERT INTO MOST_VERSATILE VALUES (NEW.PID, NEW.MOVIEID, roles);
   ```

```
    -- remove those whose number_of_roles is worse than that of more than 4 performers
    DELETE FROM MOST_VERSATILE A
    WHERE 4 < (SELECT count(*) FROM MOST_VERSATILE B
               WHERE B.number_of_roles > A.number_of_roles);
  END IF;
END;
```

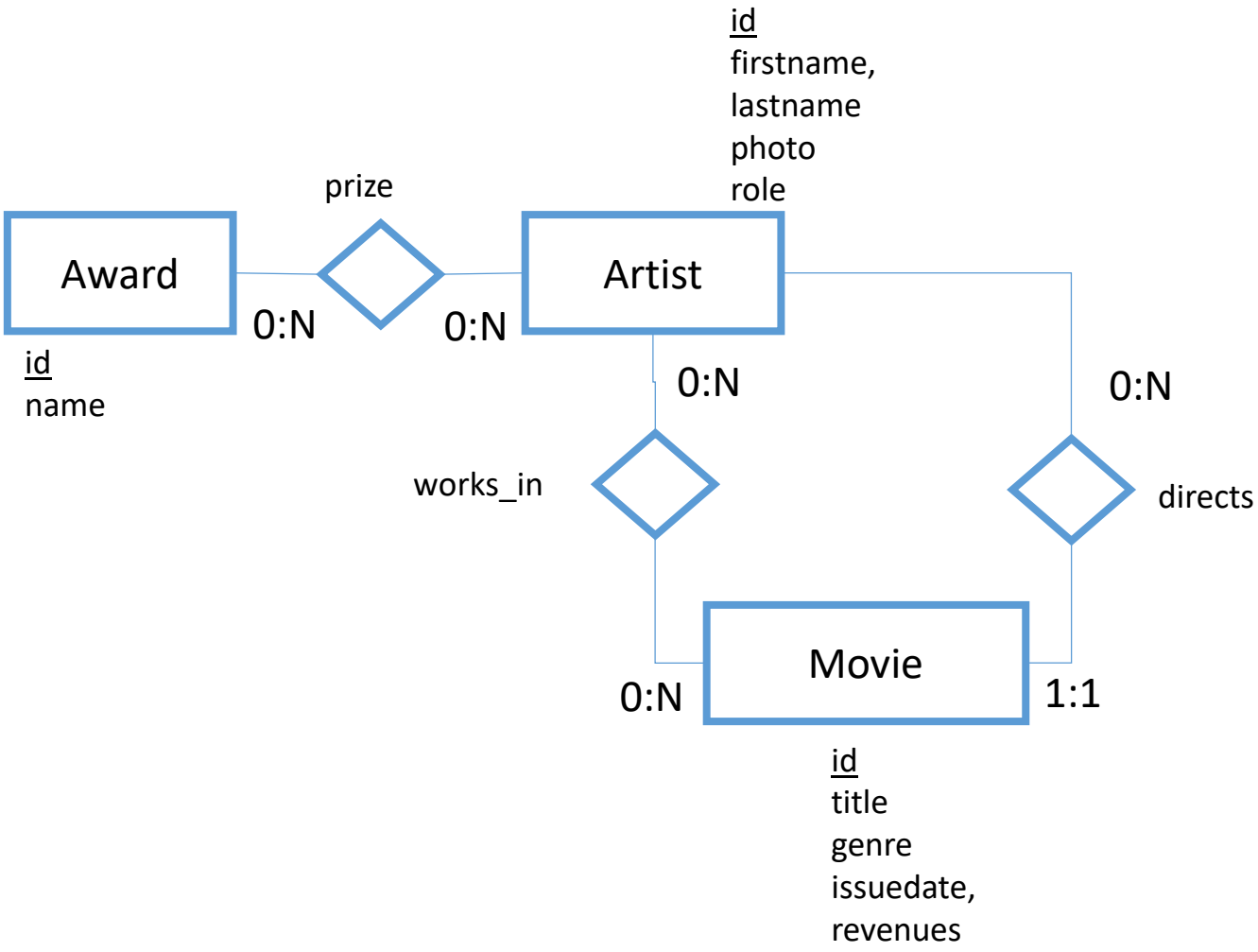
D. JPA (7 points)

An application manages a movie and actor database. A Movie has a title, an issue date, a genre and revenues (an integer number representing the gross income of the movie). An Artist can direct a Movie and/or work as an actor in a movie. Artists have a first/last name, an id, a photo, and a role (a string, e.g., comic, dramatic, etc). Artists may have won one or more awards (e.g., Oscar for direction, Oscar for best male/female actor, etc.). The application lets the user access the list of artists and browse in chronological order (descending issue date) the movies they have directed, the movies they have worked in and the list of prizes they have been awarded. The application also lets the user access the list of movies and browse the director and the cast. Movies and actors are in the order of hundreds of thousands. Awards are in the order of hundreds.

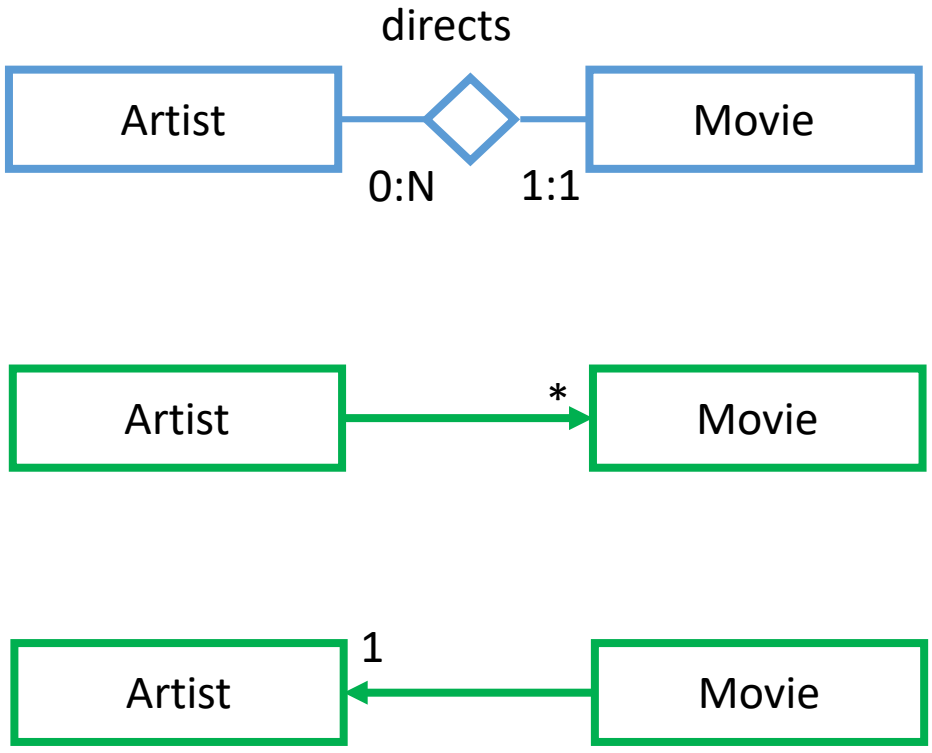


Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above-mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Entity Relationship

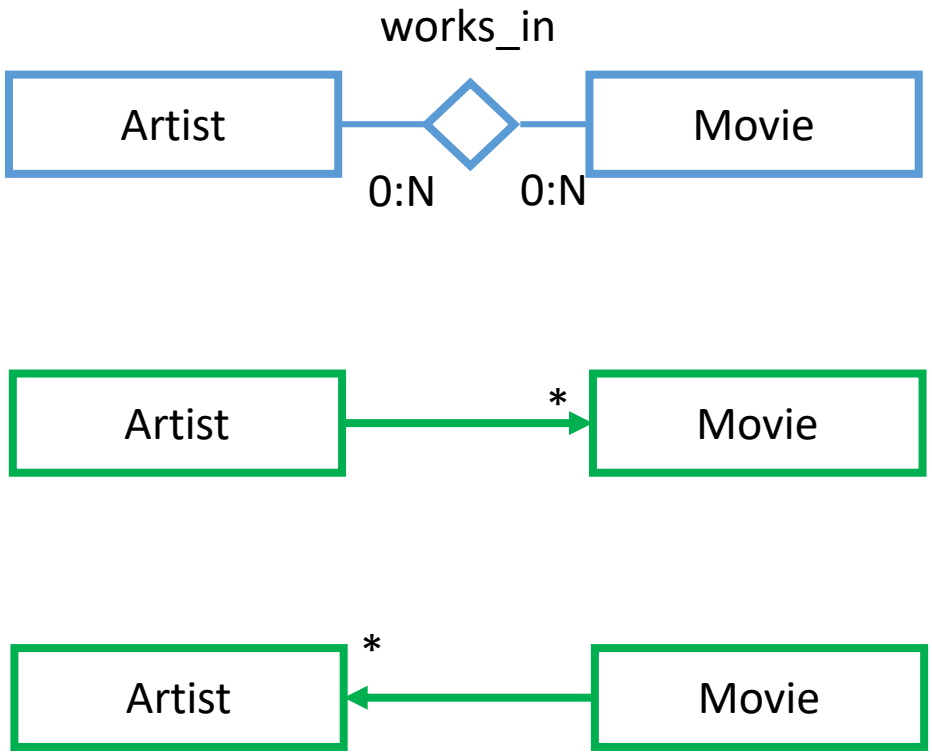


Relationship “directs”



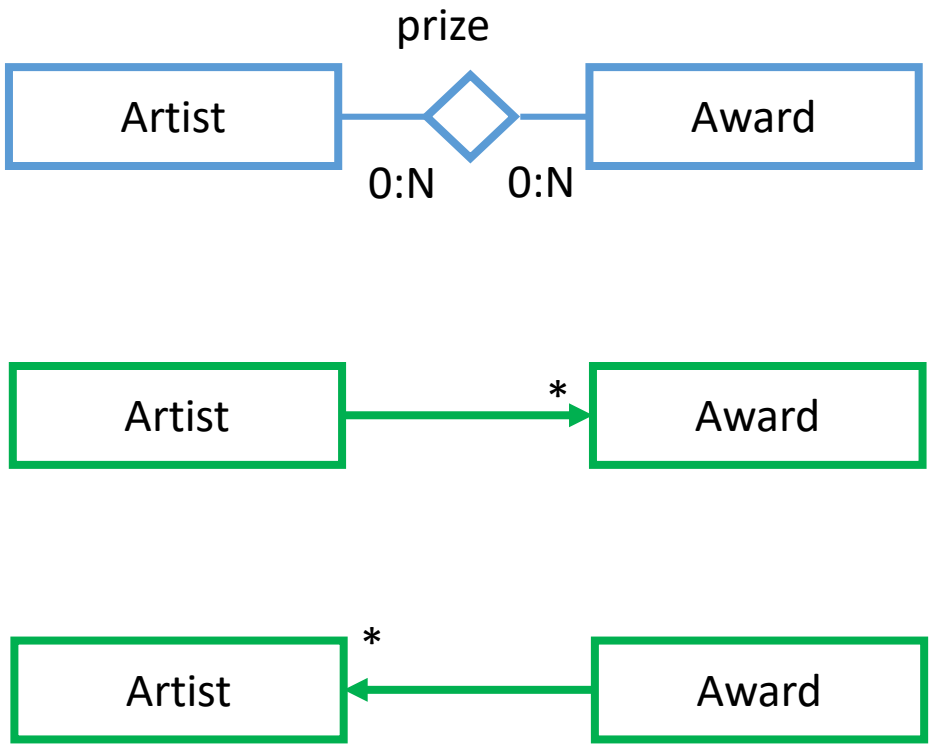
- Artist → Movie @OneToMany is necessary to get the Movies directed by the artist
  - @OrderBy(“issuedate DESC”)
  - Owner = Movie
  - FetchType.EAGER due to few movies in the directors’ curriculum
- Movie → Artist @ManyToOne is requested to show the Artist that directs it

Relationship “works_in”



- Movie → Artist @ManyToMany is necessary to show the Artists that work in a Movie
  - Owner = either Artist or Movie
  - FetchType.EAGER due to few artists in the movie cast
- Artist → Movie @ManyToMany is requested to show the list of Movies an artist works in.
  - FetchType.EAGER due to few movies in the actors’ curriculum

Relationship “prize”



- Artist → Award @ManyToMany is necessary to show the Awards of an Artist
  - Owner = either Artist or Award
  - FetchType.EAGER: few prizes per artist
- Award → Artist @ManyToMany not requested, but mapped for simplicity
  - FetchType.LAZY or FetchType.EAGER: hundreds of artists per prize

Entity Artist – version 1

```
@Entity
public class Artist implements Serializable {
	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy = GenerationType.IDENTITY)
	private int id;

	private String firstname;
	private String lastname;
	@Basic(fetch = FetchType.LAZY)
	@Lob
	private byte[] photo;
	private String role;

	@ManyToMany(mappedBy = "Artists", fetch =
	FetchType.EAGER)
	private List<Movie> participatedMovies;

	@OneToMany(mappedBy = "director", fetch =
	FetchType.EAGER)
	@OrderBy("issuedate DESC")
	private List<Movie> directedMovies;

	@ManyToMany(mappedBy = "Artists", fetch =
	FetchType.EAGER)
	private List<Award> awards;
```

Entity Movie – version 1

```
@Entity
public class Movie implements Serializable {
	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy=GenerationType.IDENTITY)
	private int id;
	private String title;
	private String genre;
	private int revenues;
	@Temporal(TemporalType.DATE)
	private Date issuedate;


	 @ManyToMany(fetch = FetchType.EAGER)
	@JoinTable(
		name="actor_movie",
		   joinColumns={@JoinColumn(name="movid")},
		   inverseJoinColumns={@JoinColumn(name="actid")}
		)
	private List<Artist> Artists; //owner

	@ManyToOne
	@JoinColumn(name="director")
	private Artist director;  //owner
	.. ..
```

Entity Artist – version 2

```
@Entity
public class Artist implements Serializable {
	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy = GenerationType.IDENTITY)
	private int id;

	private String firstname;
	private String lastname;
	@Basic(fetch = FetchType.LAZY)
	@Lob
	private byte[] photo;
	private String role;

	@ManyToMany(mappedBy = "Artists", fetch =
	FetchType.EAGER)
	private List<Movie> participatedMovies;

	@OneToMany(mappedBy = “director", fetch =
	FetchType.EAGER)
	private List<Movie> directedMovies;

	@ManyToMany(mappedBy = "Artists", fetch =
	FetchType.EAGER)
	private List<Award> awards;
```

Entity Movie – version 2

```
@Entity
public class Movie implements Serializable {
        private static final long serialVersionUID = 1L;
        @Id
        @GeneratedValue(strategy=GenerationType.IDENTITY)
        private int id;
        private String title;
        private String genre;
        private int revenues;
        @Temporal(TemporalType.DATE)
        private Date issuedate;

        @OrderBy("issuedate DESC")

        @ManyToMany(fetch = FetchType.EAGER)
        @JoinTable(
                name="actor_movie",
                   joinColumns={@JoinColumn(name="movid")},
                   inverseJoinColumns={@JoinColumn(name="actid")}
                )
        private List<Artist> Artists; //owner

        @ManyToOne
        @JoinColumn(name="director")
        private Artist director;  //owner
       .. ..
```

Entity Award

```
@Entity
public class Award implements Serializable {
	private static final long serialVersionUID = 1L;
	@Id
	@GeneratedValue(strategy=GenerationType.IDENTITY)
	private int id;
	private String name;

	@ManyToMany
	@JoinTable(
		name="art_awa",
		joinColumns={@JoinColumn(name="awid")},
		inverseJoinColumns={@JoinColumn(name="artid")}
		)
	private List<Artist> Artists; //owner
```